

UMA AULA COMPLETA, SEM PRÉ-REQUISITOS

Como este piano funciona

*A física e a matemática por trás de cada nota,
explicadas do zero, em português.*

Desenvolvimento patrocinado pela [Grabatus Labs](#)

PIANO — UM SINTETIZADOR DE PIANO FISICAMENTE MODELADO, EM RUST

COMO ESTE PIANO FUNCIONA é a versão didática, sem pré-requisitos, da documentação técnica do projeto *piano* — um sintetizador de piano fisicamente modelado, escrito em Rust, sem nenhuma amostra ou gravação de áudio real em nenhum lugar do código.

O código-fonte, a documentação técnica completa (em inglês) e este mesmo documento em Markdown estão publicados abertamente em <https://github.com/rodolphomacedo/piano>, sob licença MIT ou Apache-2.0, à escolha de quem usar.

Desenvolvimento patrocinado pela Grabatus Labs — <https://grabatus.com>.

Tipografado com L^AT_EX (motor Tectonic), nas famílias TeX Gyre Pagella, TeX Gyre Heros e TeX Gyre Cursor.

Antes de começar

Para quem é este documento? Para qualquer pessoa — inclusive quem nunca estudou física, música ou programação. Vamos construir cada ideia do zero, devagar, com exemplos do dia a dia antes de qualquer fórmula. Se em algum momento aparecer uma palavra nova (de música, física ou matemática), ela é explicada *antes* de ser usada, com uma caixa como esta:

PALAVRA NOVA. Explicação simples, com exemplo.

Não existe pré-requisito. Se você conseguiu ler até aqui, consegue ler o documento inteiro.

Sumário

Antes de começar	ii
1 O que este projeto realmente faz	1
2 O que é som, afinal	2
3 O que é uma nota musical	3
4 Como um piano de verdade produz som	4
5 Por que instrumentos diferentes soam diferentes: harmônicos e timbre	5
6 Como um computador “ouve” e “fala” som	6
7 A grande sacada: uma corda é uma fila de espera	7
8 Por que a nota morre: o filtro que come agudos	11
9 O ajuste fino: por que o laço é um pouquinho mais curto	14
10 Cordas de verdade são rígidas: a inarmonicidade	15
11 O martelo de feltro: como uma martelada vira som	17
12 Mais de uma corda por tecla: uníssono e batimento	19
13 O pedal mágico: ressonância simpática	21
14 A caixa de ressonância: de onde vem o volume	22
15 Por que o programa nunca pode travar	23
16 Como as peças do projeto se encaixam	24
17 O que ainda falta para chegar perto de um piano de verdade	25
18 Tocando de verdade: como o MIDI leva sua martelada até o código	26
Glossário completo	30
Para saber mais	31

O que este projeto realmente faz

EXISTEM dois jeitos completamente diferentes de fazer um “piano digital” tocar uma nota. **Jeito 1 — gravar e reproduzir.** Alguém senta em um piano de verdade, grava cada tecla sendo tocada em vários volumes diferentes, e guarda milhares de arquivos de áudio. Quando você aperta uma tecla no teclado digital, ele simplesmente *toca de volta* a gravação certa. É assim que a imensa maioria dos pianos digitais, teclados e aplicativos de celular funciona — isso se chama *sampler*.

SAMPLER. Um instrumento que reproduz gravações prontas em vez de calcular o som.

Funciona bem, mas tem um problema conceitual: o piano nunca está “tocando” de verdade, ele está apenas apertando o “play” numa gravação antiga.

Jeito 2 — calcular a física. É o que este projeto faz. Dentro do código não existe nenhum arquivo de áudio, nenhuma gravação, nenhum “play”. Existe uma simulação matemática de como uma corda de piano de verdade se comporta: como ela vibra quando é golpeada, como perde energia com o tempo, como duas ou três cordas da mesma tecla interagem, como a caixa de madeira do piano colore o som. Cada vez que você aperta uma tecla, o programa **calcula**, do zero, amostra por amostra, 48 mil vezes a cada segundo, o que uma corda de piano de verdade faria fisicamente. Isso se chama *modelagem física*.

MODELAGEM FÍSICA. (*physical modelling*, em inglês) Descrever um objeto do mundo real — aqui, uma corda de piano — como um conjunto de equações, e resolver essas equações em tempo real para produzir o comportamento do objeto, em vez de gravar o objeto.

Essa é a razão de existir de todo este documento: explicar, devagar e com exemplos, **como é possível calcular o som de uma corda de piano em tempo real, e quais pedacinhos da física real** este projeto já modelou (e quais ainda faltam).

Nada aqui é segredo ou “mágica de IA”. É física do século XIX e XX (a mesma que rege cordas de violão, o balanço de um parquinho e as ondas do mar), descrita como fórmulas, e as fórmulas transformadas em código Rust que roda rápido o suficiente para acontecer ao vivo.

O que é som, afinal

ANTES de falar de piano, precisamos entender o que é som — porque tudo o que vem depois é, no fundo, sobre como *fabricar* som a partir de números.

Bata palmas. O que aconteceu, fisicamente? Suas mãos empurraram o ar que estava entre elas. Esse ar empurrado empurrou o ar vizinho, que empurrou o próximo, e assim por diante — uma onda de pressão se espalhando pelo ambiente, exatamente como quando você joga uma pedra num lago parado e vê os círculos se espalharem na água.

ONDA. Uma perturbação que se propaga através de um meio (ar, água, uma corda) sem que o próprio meio “viaje” — é a *perturbação* que anda, não o ar em si. Pense em uma ola de estádio: cada torcedor só levanta e senta no próprio lugar, mas o movimento parece “correr” pela arquibancada.

Quando essa onda de pressão chega até o seu ouvido, ela empurra e puxa o tímpano — uma membrana bem fininha, como a pele esticada de um tambor — para frente e para trás, muito rapidamente. O cérebro interpreta esse vaivém-vaivém-vaivém como som.

A pergunta importante é: **com que velocidade** o ar empurra e puxa? Se você bater palmas devagar (uma vez por segundo), o ar até vibra, mas tão devagar que você não ouviria “som” nenhum — ouviria só um “bump” isolado. Para existir uma nota musical, o vaivém precisa se repetir *centenas ou milhares de vezes por segundo*. É disso que o próximo capítulo trata.

O que é uma nota musical

FREQUÊNCIA. Quantas vezes por segundo algo se repete. Se seu coração bate 70 vezes por minuto, a frequência dos seus batimentos é “70 por minuto”. Para som, medimos frequência em **hertz** (abreviado **Hz**), que quer dizer “vezes por segundo”. Um som de 440 Hz vibra o ar para frente e para trás 440 vezes a cada segundo.

UMA nota musical nada mais é do que um som cuja frequência é constante e bem definida — o vaivém do ar se repete no mesmo ritmo, sem parar, durante o tempo em que a nota soa. Quanto **mais alta** a frequência, mais **aguda** (fina) a nota soa aos nossos ouvidos; quanto **mais baixa**, mais **grave** (grossa).

Um piano tem 88 teclas, e cada uma corresponde a uma frequência diferente. A tecla mais grave (a mais à esquerda, chamada **A0**, ou **Lá0**) vibra a 27,5 Hz. A tecla mais aguda (a mais à direita, **C8**, ou **Dó8**) vibra a 4186 Hz — mais de 150 vezes mais rápido. A nota de referência que afinadores de piano (e este projeto) usam como ponto de partida é o **A4** (**Lá4**, o Lá “do meio”), fixado internacionalmente em **440 Hz**.

OITAVA. Duas notas estão “uma oitava” de distância quando a frequência de uma é exatamente o dobro da outra. A4 é 440 Hz; uma oitava acima, A5, é 880 Hz; uma oitava abaixo, A3, é 220 Hz. É por isso que dobrar (ou cortar pela metade) a frequência soa, aos nossos ouvidos, como “a mesma nota, só que mais aguda/grave” — não é coincidência, é como o ouvido humano processa proporções de frequência.

NOTAÇÃO DE NOTA + OITAVA (EX. A4, C3, F#5). A letra é o nome da nota (A, B, C, D, E, F, G — o equivalente em inglês de Lá, Si, Dó, Ré, Mi, Fá, Sol) e o número indica em qual oitava ela está — quanto maior o número, mais aguda. Um # (sustenido) depois da letra significa “meio tom acima”. Este projeto (e a maior parte do software musical no mundo) usa essa notação em inglês, então “A4” é o Lá central e “C4” é o Dó central.

O piano é um instrumento **temperado**: o espaço entre A0 e C8 é dividido em 88 degraus (chamados **semitons**) espaçados de um jeito matematicamente preciso, onde cada semitom multiplica a frequência anterior por aproximadamente 1,0595 (a raiz 12ª de 2 — porque 12 semitons formam uma oitava, e uma oitava dobra a frequência: $1,0595^{12} \approx 2$). Você não precisa decorar esse número; o que importa entender é que **cada tecla do piano é uma frequência-alvo específica e fixa**, e é exatamente essa frequência-alvo que o programa recebe quando você aperta uma tecla — o trabalho de todo o resto deste documento é: *dada essa frequência, como calcular o som de uma corda vibrando nela?*

Como um piano de verdade produz som

ABRA a tampa de um piano de verdade (ou imagine um cortado ao meio) e você vai ver, para a maioria das teclas, isto:

1. Você aperta uma tecla.
2. Um mecanismo mecânico (a **ação**) lança um pequeno **martelo coberto de feltro** contra uma ou mais **cordas de aço** esticadas sob tensão.
3. O martelo bate na corda e volta — ele não fica encostado, é uma martelada rápida, de menos de 5 milésimos de segundo.
4. A corda, agora vibrando, empurra a **caixa de ressonância** (o grande tampo de madeira embaixo/atrás das cordas) através de uma peça de madeira chamada **ponte**.
5. A caixa de ressonância, por ser grande e leve, empurra o ar de verdade e é o que você realmente ouve — a corda sozinha, fininha como é, mal move ar suficiente para ser audível.
6. Enquanto a tecla continua pressionada, a corda continua vibrando e perdendo energia aos poucos, até o som sumir. Quando você solta a tecla, um **abafador** (*damper*) de feltro encosta na corda e para a vibração imediatamente.
7. Se você pisar no **pedal direito** (pedal de sustentação, também chamado de *sustain*), todos os abafadores do piano levantam ao mesmo tempo — a corda que você tocou continua soando mesmo depois de soltar a tecla, e todas as outras cordas do piano ficam livres para vibrar também, se algo as excitar (capítulo 13).

PONTE. (*bridge*) A peça que transmite a vibração da corda para a caixa de ressonância, como o “sela” de um violão.

Esse é o piano inteiro, em sete passos. Cada capítulo a partir daqui pega um desses passos e explica: (a) a física real por trás dele, e (b) como este projeto o transforma em cálculo matemático que um computador consegue fazer 48 mil vezes por segundo.

Por que instrumentos diferentes soam diferentes: harmônicos e timbre

TOQUE a mesma nota — digamos, A4 (440 Hz) — num piano e num violino. Ambos vibram a 440 Hz. Ambos são, estritamente, “a mesma nota”. E ainda assim eles soam completamente diferentes. Por quê?

A resposta é que **nenhuma corda vibra em apenas uma frequência**. Quando uma corda é golpeada ou puxada, ela vibra simultaneamente:

- inteira, de ponta a ponta (essa é a **frequência fundamental** — a nota “principal” que você identifica, 440 Hz no nosso exemplo);
- em duas metades, cada metade vibrando ao dobro da velocidade (880 Hz);
- em três terços, cada terço vibrando ao triplo da velocidade (1320 Hz);
- e assim por diante, em infinitas divisões cada vez mais fracas.

HARMÔNICO. (ou *parcial*, ou *overtone*) Cada uma dessas vibrações simultâneas de uma corda. O primeiro harmônico (a corda inteira) é a fundamental; o segundo, terceiro, quarto harmônico etc. são múltiplos inteiros dela ($2\times$, $3\times$, $4\times$ a frequência fundamental) e em geral soam mais fracos quanto mais alto o número.

TIMBRE. A “cor” ou “textura” característica de um instrumento — aquilo que faz um piano soar diferente de um violino tocando a mesma nota. O timbre é determinado principalmente pela **proporção de volume entre os harmônicos**: um piano tem harmônicos numa certa proporção, um violino tem em outra, e é essa “receita” de harmônicos que seu ouvido reconhece como “isto é um piano”.

Todo o trabalho de simular um instrumento de verdade se resume, em grande parte, a produzir a *mistura certa de harmônicos, no volume certo, morrendo na velocidade certa* — porque é essa mistura que faz a diferença entre “um bipe eletrônico monótono” e “aquilo soa como um piano de verdade”. Os capítulos 9 a 14 mostram, um a um, os mecanismos físicos reais que moldam essa mistura num piano de verdade — e como este projeto os reproduz.

Como um computador “ouve” e “fala” som

Um computador não entende “ondas contínuas de pressão do ar” — ele só sabe guardar e processar números. Para representar som digitalmente, fazemos o seguinte: **medimos a pressão do ar (ou, equivalentemente, a posição de um alto-falante) muitas vezes por segundo, e guardamos cada medida como um número.**

AMOSTRA. (*sample*) Um único número que representa “qual era a pressão do ar (ou posição do alto-falante) neste instante exato”. Uma amostra sozinha não tem som nenhum — é só um número, como uma única foto de um vídeo.

TAXA DE AMOSTRAGEM. (*sample rate*) Quantas amostras são medidas (ou geradas) por segundo. Este projeto usa **48.000 amostras por segundo** (48 kHz) — o mesmo padrão usado em vídeo profissional e na maioria dos equipamentos de áudio modernos. Isso significa: para produzir 1 segundo de som, o programa precisa calcular 48.000 números, em sequência, um após o outro.

Por que 48.000 e não, digamos, 100? Porque, para reconstituir uma onda com fidelidade, é preciso amostrá-la a uma taxa **pelo menos duas vezes maior** que a frequência mais alta que você quer representar (isso é um resultado matemático chamado Teorema de Nyquist–Shannon). O ouvido humano escuta até cerca de 20.000 Hz; 48.000 amostras por segundo dão folga confortável acima disso.

Isso também nos dá um número muito importante para os próximos capítulos: a cada amostra, o programa tem $\frac{1}{48000}$ de segundo — cerca de **20,8 microssegundos** (0,0000208 segundos) — para calcular o próximo número antes que ele precise ser enviado à placa de som. Repita isso 48 mil vezes, e você tem um segundo de piano tocando. É essa corrida contra o tempo, repetida sem parar, que faz o capítulo 15 (sobre o programa “nunca poder travar”) ser tão importante.

A grande sacada: uma corda é uma fila de espera

CHEGAMOS à ideia central do projeto inteiro — a peça de engenharia que torna possível simular uma corda de piano em tempo real, sem precisar de um supercomputador.

A analogia do eco no corredor

Imagine um corredor comprido, com uma parede em cada ponta. Você grita uma vez na entrada. O som viaja até a parede do fundo, ecoa, volta, ecoa de novo na parede de onde você está, volta a ir... e assim por diante, ficando mais fraco a cada ida e volta, até sumir. O que você ouve, com o tempo, é uma série de ecos regularmente espaçados, cada um mais fraco que o anterior.

Uma corda de piano vibrando é **exatamente esse fenômeno**, só que muito mais rápido: quando o martelo bate na corda, ele cria uma perturbação que viaja de uma ponta da corda até a outra, reflete, volta, reflete de novo — e esse “ir e vir” que se repete centenas ou milhares de vezes por segundo é o que você ouve como uma nota musical. A física formal por trás disso (a “equação de onda”) mostra algo bonito: a vibração de uma corda pode sempre ser descrita como **duas ondas viajando em direções opostas**, quicando entre as duas pontas fixas.

Por que não gravar um único ciclo e repeti-lo?

Antes de seguir, vale examinar de frente uma ideia alternativa, bem razoável à primeira vista: “por que não simplesmente calcular a forma de uma vibração completa da corda — um ciclo já com todos os harmônicos embutidos — e repetir esse mesmo pedacinho de onda sem parar, na taxa de amostragem, pelo tempo que a nota durar? Para a nota morrer, bastaria ir aplicando por cima um filtro que vai diminuindo o volume”. Essa técnica existe de verdade — chama-se *wavetable synthesis*, “síntese por tabela de onda” — e é usada por muitos instrumentos eletrônicos. Ela não é usada aqui, e a razão é reveladora sobre o que este projeto realmente simula.

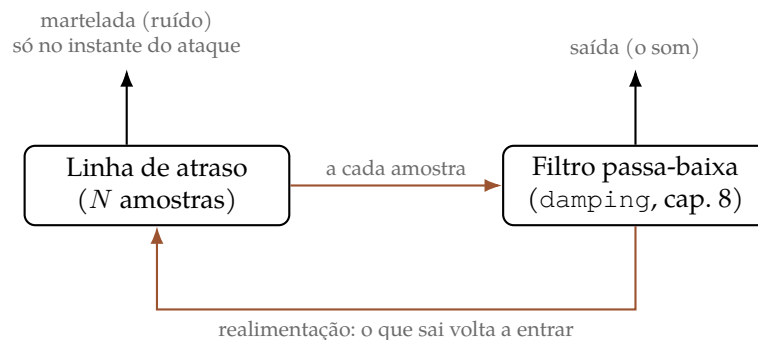
O problema: qual ciclo, exatamente, eu repito?

Para repetir “um ciclo com todos os harmônicos já embutidos”, é preciso primeiro *saber* qual é esse ciclo — sua forma exata já teria que estar pronta antes de começar. Mas o capítulo 8 mostra um fato central sobre como um piano de verdade soa: **o ciclo não é sempre o mesmo**. No instante em que o martelo bate, a onda é rica em harmônicos agudos (um formato “pontudo”, cheio de detalhe fino); alguns segundos depois, os agudos já morreram e sobrou uma onda bem mais “redonda”, próxima de um seno puro (capítulo 8 explica exatamente por quê). Repetir sempre o mesmo ciclo geraria um som estático, sem essa evolução — e é exatamente por isso que um sintetizador *wavetable* simples costuma soar “eletrônico” em vez de “de verdade”: ele reproduz uma foto congelada do timbre, não uma corda perdendo energia continuamente e de forma diferente em cada frequência.

A saída óbvia seria: então calcule uma tabela de onda *diferente* para cada instante da nota, já com o timbre certo daquele momento — mas isso é, palavra por palavra, o problema inteiro que este projeto resolve. Seria preciso já conhecer a resposta (o som completo, evoluindo no tempo) antes de sintetizá-la.

A saída: deixar a física calcular a evolução sozinha

A linha de atraso resolve isso invertendo o problema. Em vez de armazenar *o resultado* (a onda já pronta, em qualquer instante), ela armazena *o mecanismo físico* que produz esse resultado: uma perturbação que viaja, ida e volta, perdendo um pouquinho de agudo a cada volta (capítulo 8). O timbre evoluindo com o tempo — pontudo no ataque, redondo no final — não precisa ser calculado à parte, nem precisa já ser conhecido de antemão: ele *emerge* sozinho, volta após volta, do mesmo laço simples repetido milhares de vezes por segundo. É a diferença entre gravar um filme inteiro do início ao fim e simular, quadro a quadro, as leis de física que o produziriam — a segunda opção é mais barata, se generaliza sozinha para qualquer corda (grave, aguda, martelada fraca ou forte) sem gravar nada novo, e é fisicamente a coisa certa a fazer, porque é literalmente o que uma corda de verdade faz.



O QUE ESTE DIAGRAMA DIZ. Não existe, nessa figura, nenhum lugar onde o “formato da onda” esteja guardado por inteiro. Existe só um número circulando entre duas operações simples — avançar na fila, perder um pouco de agudo — repetidas milhões de vezes ao longo de uma nota. É essa repetição, não uma tabela pronta, que produz o som e a sua evolução no tempo.

Transformando isso em código: a linha de atraso

Esse fato — “é só uma onda indo e voltando” — é ótimo porque uma “onda indo e voltando” é exatamente o que um computador já sabe representar naturalmente: uma **fila de espera com um número fixo de posições**, onde a cada passo (a cada amostra) o número mais antigo sai por um lado, todos os outros andam uma posição, e um número novo entra pelo outro lado. Isso, em programação, se chama **linha de atraso** (*delay line*).

LINHA DE ATRASO. (*delay line*) Uma fila de números de tamanho fixo. A cada passo, um número novo entra, e o número mais antigo sai — como uma fila de pessoas esperando ônibus, onde uma pessoa nova entra na fila atrás e a da frente sobe no ônibus a cada minuto.

Em vez de calcular “qual é a altura de cada pontinho da corda, em cada instante” (o que exigiria recalculer *milhares* de pontos, 48 mil vezes por segundo — caro demais até para computadores modernos, se você tem 88 teclas tocando ao mesmo tempo), o projeto guarda só **uma fila circulando**, do tamanho certo, representando a onda viajando de ida e volta pela

corda. Isso é chamado de **guia de onda digital** (*digital waveguide* — a técnica, publicada por Kevin Karplus e Alex Strong em 1983 e refinada depois por Julius O. Smith, é a base de todo `piano_core`, o coração deste projeto).

O custo computacional de processar essa fila é **o mesmo, não importa quão grave ou aguda seja a corda** — é isso que torna possível tocar as 88 teclas de um piano ao mesmo tempo em tempo real, mesmo num laptop comum.

Quantas posições tem a fila?

Aqui é onde a frequência da nota (capítulo 3) entra na conta. Uma onda completa “ida e volta” pela corda corresponde a exatamente um ciclo da vibração. Então o tamanho da fila (quantas amostras ela precisa guardar) é:

$$\text{tamanho da fila} = \frac{\text{taxa de amostragem}}{\text{frequência da nota}}$$

Para A4 (440 Hz), a 48.000 amostras por segundo:

$$\text{tamanho da fila} = \frac{48\,000}{440} \approx 109,1 \text{ amostras}$$

Ou seja: a fila que representa a corda de A4 tem pouco mais de 109 posições. Para a nota mais grave do piano, A0 (27,5 Hz), a fila tem $\frac{48\,000}{27,5} \approx 1745$ posições — uma fila bem mais longa, porque uma corda grave é (física e metaforicamente) “mais comprida” e o som demora mais para dar a volta completa.

Repare que o número não deu exatamente 109 — deu 109,1. Essa fração é importante, e o capítulo 9 explica por quê (e como o projeto lida com ela sem desafinar a nota).

De onde vêm os harmônicos, na prática: nenhum é calculado separadamente

Vale parar aqui e responder uma pergunta que essa fila de espera levanta: os harmônicos do capítulo 5 — a fundamental, o dobro, o triplo, e por aí em diante — de onde eles saem, exatamente, no código? A resposta é a parte mais elegante da ideia inteira: **eles não são calculados um por um**. Não existe, em lugar nenhum de `piano_core`, um laço do tipo “gere o harmônico 1, depois o harmônico 2, depois o harmônico 3”.

O que existe é só a fila de espera realimentando a si mesma, do jeito descrito acima. E uma fila de tamanho fixo realimentada assim tem, por pura matemática (nenhuma física adicional é necessária para chegar nesse resultado — é uma propriedade de qualquer laço de atraso realimentado, o mesmo princípio por trás do que processamento de sinais chama de “filtro pente”, *comb filter*), ressonância exatamente na frequência fundamental *e em todos os seus múltiplos inteiros ao mesmo tempo* — $2\times$, $3\times$, $4\times$, e assim sucessivamente, até o limite que a taxa de amostragem permite representar (capítulo 6). Excitar essa fila com ruído (capítulo 11) equivale a “cutucar” todas essas ressonâncias de uma vez; a fila devolve, sozinha, energia amplificada em cada uma delas, na proporção que a física de cada uma dita — sem que nenhuma linha de código precise saber que o harmônico 7 existe.

POR QUE ISSO IMPORTA. Esse é o motivo pelo qual este projeto não precisa — e não usa — uma técnica chamada **síntese aditiva**, que soma manualmente um oscilador senoidal para cada harmônico desejado (um para a fundamental, outro para o dobro, outro para o triplo...). Síntese aditiva é uma abordagem legítima, usada por muitos sintetizadores, mas é uma técnica *diferente*: em vez de simular a física de uma corda, ela reconstrói o resultado sonoro “de fora para dentro”, harmônico por harmônico, escolhido à mão. Este projeto simula a corda; os harmônicos são uma *consequência* do modelo físico, não uma entrada dele.

“Dá para eu escolher ter só 1 ou 4 harmônicos?”

Não, não do jeito que a pergunta sugere — e vale explicar o porquê, em vez de simplesmente dizer não. Numa síntese aditiva, “quantos harmônicos” é literalmente um parâmetro: você soma quantos osciladores quiser. Neste projeto essa alavanca não existe porque a arquitetura é outra: os harmônicos não são *itens de uma lista* que o código gera um a um e que poderiam, por isso, ser cortados de uma lista mais curta. Eles são *o espectro de ressonância de uma fila de espera realimentada* — e uma fila realimentada sempre ressoa no conjunto inteiro de múltiplos da fundamental, nunca só nos que alguém escolheu à mão, pela mesma razão física que uma corda de violão de verdade não deixa você “desligar” o terceiro harmônico sem desligar a corda inteira.

O que existe, de verdade, são três alavancas que mudam *quanto* sobra de cada harmônico, sem nunca eliminar um harmônico específico:

- **damping** (capítulo 8): quanto mais alto, mais depressa os harmônicos agudos morrem em relação ao grave — levado ao extremo, a nota soa quase só como a fundamental depois de poucos milissegundos, mesmo que todos os harmônicos tenham sido excitados no ataque.
- **inharmonicidade** (capítulo 10): desloca a *posição* de cada harmônico (o capítulo 10 explica quanto), não a sua presença ou ausência.
- **a força da martelada** (capítulo 11, via `piano_core::hammer::HammerConfig`): muda o *equilíbrio espectral* da excitação inicial — uma martelada mais dura acende os harmônicos agudos com mais energia relativa — mas continua acendendo todos eles, nunca só um subconjunto escolhido.

Há também um limite físico embutido, não uma escolha do usuário: nenhum harmônico acima da metade da taxa de amostragem (o limite de Nyquist, capítulo 6) pode existir de jeito nenhum — a 48 000 amostras por segundo, o teto é 24 000 Hz. Para A4 (440 Hz) isso deixa espaço para cerca de 54 múltiplos inteiros antes do teto; para A0 (27,5 Hz), quase 870 — embora, na prática, o filtro passa-baixa do capítulo 8 já tenha silenciado a esmagadora maioria deles bem antes de o ouvido humano (que também para de ouvir por volta de 20 000 Hz) ter qualquer chance de notar.

O resto do laço

Uma fila sozinha, recirculando um número para sempre, tocaria a nota **para sempre**, sem nunca morrer — o que não é o que acontece numa corda de verdade. Faltam ainda duas peças, que os próximos dois capítulos explicam: algo que **tire energia** da fila a cada volta (o som precisa morrer com o tempo, capítulo 8), e algo que **inicie** a vibração de um jeito parecido com uma martelada de feltro, não um empurrão genérico (capítulo 11).

Por que a nota morre: o filtro que come agudos

NUMA corda ideal e sem atrito, a energia da martelada ficaria circulando na fila para sempre — a nota tocaria infinitamente, o que obviamente não é o que acontece num piano de verdade. Na vida real, a corda perde energia a cada ida e volta: um pouco escapa pela ponte para a caixa de ressonância (de propósito — é assim que você ouve o som!), e um pouco se perde em atrito interno do próprio metal e do ar ao redor.

O detalhe físico interessante — e crucial para soar como um piano de verdade, não como um bipe genérico — é que **essa perda de energia não é igual para todos os harmônicos**. Os harmônicos mais agudos (as vibrações mais rápidas, capítulo 5) perdem energia muito mais depressa que os graves. É por isso que uma nota de piano começa com um ataque brilhante, cheio de agudos, e vai ficando cada vez mais “redonda” e grave conforme soa — os agudos vão embora primeiro, sobra só o fundamental por último.

FILTRO PASSA-BAIXA. (*lowpass filter*) Um mecanismo que deixa frequências baixas passarem quase sem mudança, mas reduz (atenua) frequências altas. É exatamente o oposto de um cobertor grosso jogado sobre uma caixa de som: os graves (o “bum bum” do baixo) você ainda ouve do lado de fora, os agudos (o “tss tss” do prato de bateria) somem quase por completo — o cobertor está agindo como um filtro passa-baixa físico.

A solução do projeto: dentro da fila de espera (a linha de atraso do capítulo 7), antes do número voltar a circular, ele passa por um filtro passa-baixa simples — implementado como `filter::OnePoleLowpass` no código — que reduz um pouquinho a “agudeza” do número a cada volta.

“UM POLO” (ONE-POLE). A versão mais simples possível de um filtro — precisa de só uma multiplicação e uma soma por amostra para funcionar. “Polo” é um termo técnico de processamento de sinais; o que importa saber é que, apesar de simples, um filtro de um polo já é suficiente para capturar o comportamento essencial de “agudos morrem mais rápido que graves” que uma corda de verdade exhibe.

O que o filtro realmente faz, amostra a amostra

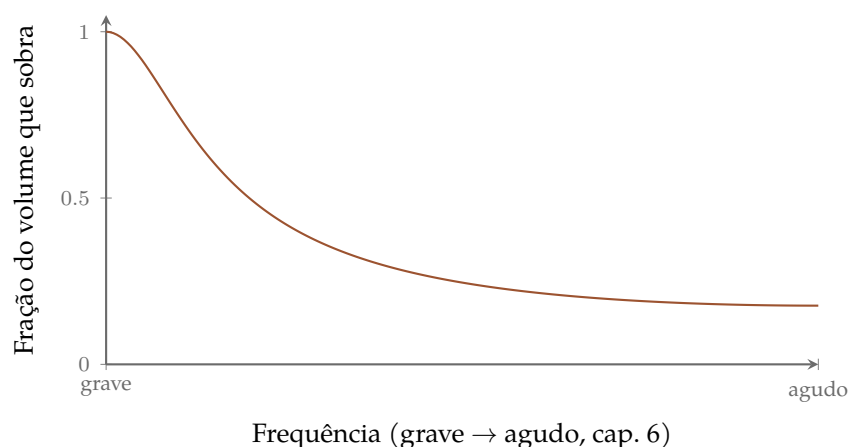
Concretamente, o filtro guarda um único número — sua própria saída na amostra anterior — e, a cada amostra nova, mistura um pouco do valor que acabou de chegar com um pouco do que ele já tinha:

$$\text{saída}_i = (1 - a) \cdot \text{entrada}_i + a \cdot \text{saída}_{i-1}$$

O número a (entre 0 e 1, mais perto de 1 quanto maior o *damping*) é o quanto o filtro “gruda” no valor anterior. Se a fosse 0, o filtro copiaria a entrada sem mudar nada; se a fosse quase 1, a saída mal se move de uma amostra para a outra — o que, num sinal que sobe e desce rapidamente (um agudo), apaga quase todo o movimento, e num sinal que sobe e desce devagar (um grave), deixa passar quase tudo. É essa única linha de matemática, repetida amostra a amostra, que produz o comportamento “deixa passar grave, apaga agudo”

descrito acima.

O gráfico abaixo mostra exatamente isso: quanto do volume original de cada frequência sobra depois de passar pelo filtro **uma única vez**, para um valor de a típico.



COMO LER ESTE GRÁFICO. Perto de “grave” (esquerda), a curva está em 1: o filtro deixa passar praticamente 100 % do volume. Perto de “agudo” (direita), a curva cai para perto de zero: o filtro apaga quase tudo. Isso é o efeito de **uma única passagem** — pouco dramático sozinho. O que faz toda a diferença é repetir essa mesma passagem milhares de vezes por segundo, uma a cada volta da fila de espera — é isso que a próxima seção detalha.

“É a cada segundo que ele aplica o damping?”

Esta é a confusão mais comum ao pensar nesse mecanismo pela primeira vez, então vale ser bem explícito: **o filtro roda em toda e qualquer amostra**, sempre — 48 000 vezes por segundo, para qualquer nota, grave ou aguda, sem exceção nenhuma. Ele não “espera um segundo” para agir, nem age uma vez por nota: ele age em cada uma das 48 000 amostras que o computador calcula a cada segundo (capítulo 6).

O que muda de nota para nota é **quantas voltas completas** o sinal dá na fila (capítulo 7) dentro de 1 segundo — e cada volta completa passa pelo filtro exatamente uma vez, porque o filtro fica dentro do laço, não fora dele. Uma volta completa é, por definição, um ciclo da nota (capítulo 3), então:

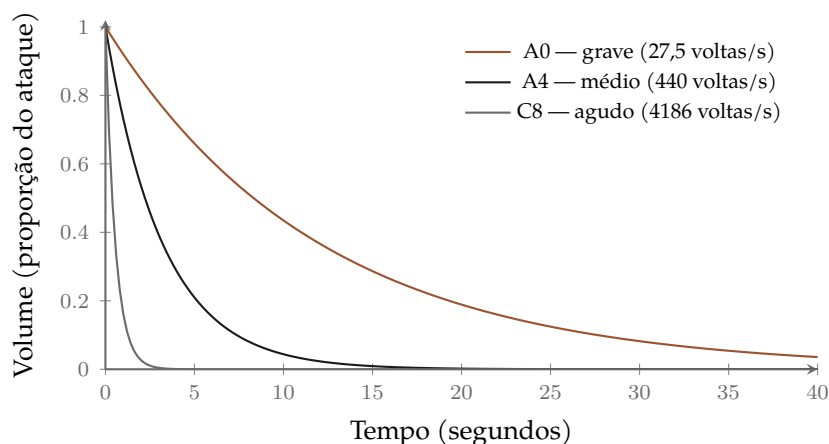
$$\text{voltas por segundo} = \text{frequência da nota}$$

Uma corda de A4 (440 Hz) dá 440 voltas completas por segundo — passa pelo filtro 440 vezes por segundo. Uma corda de A0 (27,5 Hz) dá só 27,5 voltas por segundo — passa pelo filtro 27,5 vezes por segundo, **dezesseis vezes menos frequentemente que A4**, mesmo com o *mesmo* damping. Essa diferença — não uma diferença física maior no metal da corda — é a razão principal pela qual cordas graves soam por dezenas de segundos e cordas agudas por 1 ou 2: a corda grave não perde menos energia por volta, ela só dá muito menos voltas por segundo, então a mesma perda por volta demora muito mais tempo para se acumular.

Nota	Frequência	Passagens pelo filtro / segundo	Decaimento típico
A0 (grave)	27,5 Hz	27,5	30–40 s
A4 (médio)	440 Hz	440	8–15 s
C8 (agudo)	4186 Hz	4186	1–2 s

HONESTIDADE SOBRE A TABELA. Os decaimentos típicos não vêm só das voltas por segundo — *damping* e *sustain* também variam um pouco por registro no código, calibrados para casar com a tabela medida em `docs/PHYSICS.md`. O ponto central continua de pé: a *mesma* perda por volta produz decaimentos muito diferentes conforme a nota, só porque o número de voltas por segundo já é, sozinho, muito diferente.

O resultado acumulado — milhares (ou dezenas de milhares) de pequenas perdas de agudo, uma a cada volta, repetidas pelo tempo que a nota durar — é uma curva de queda de volume aproximadamente exponencial: ela nunca “zera de repente”, cai bastante nos primeiros instantes em termos absolutos e cada vez mais devagar depois. O gráfico abaixo mostra essa curva para as três notas da tabela acima, cada uma no seu próprio ritmo de voltas por segundo.



O tanto de energia que se perde a cada volta — o valor de a da fórmula acima — é controlado, no código, por um parâmetro chamado *damping*.

AMORTECIMENTO. (*damping*) O quanto de energia uma corda perde por unidade de tempo; mais amortecimento = a nota morre mais rápido.

É esse número — junto de um segundo parâmetro, *sustain* — que faz uma corda grave soar por 30 a 40 segundos e uma corda aguda por apenas 1 a 2 segundos, exatamente como num piano de verdade.

O ajuste fino: por que o laço é um pouquinho mais curto

VOLTANDO ao capítulo 7: calculamos que a fila para A4 devia ter cerca de 109,1 posições. Só que uma fila de espera só pode ter um número **inteiro** de posições — não existem “0,1 de uma posição”. Se simplesmente arredondássemos para 109, a nota tocaria ligeiramente mais aguda que 440 Hz; se arredondássemos para cima, ligeiramente mais grave. Qualquer um dos dois desafinaria o piano — pouco nas notas graves, perceptivelmente nas agudas (quanto mais aguda a nota, menor é a fila, e um erro de “1 posição inteira” pesa proporcionalmente mais).

A solução tecnicamente correta é permitir uma fila de tamanho **fracionário** — por exemplo, calcular a posição “109,1” como uma média ponderada entre a posição 109 e a posição 110. Esse pedacinho de matemática (chamado **interpolação**) é o que garante que qualquer nota, não só as que “dão números redondos” de amostras, saia afinada.

Tem ainda uma segunda sutileza. O filtro passa-baixa do capítulo 8 não é instantâneo — ele próprio introduz um pequeno atraso extra no sinal (assim como esperar um cobertor “amortecer” um som leva uma fração de segundo, não é instantâneo). Se esse atraso extra não fosse descontado, o laço inteiro ficaria um pouquinho mais longo do que deveria, e a nota sairia ligeiramente mais grave do que a frequência pedida. A correção é simples de enunciar, ainda que a matemática exata do “quanto” venha de como o filtro é construído:

$$\text{tamanho real da fila} = \text{período da nota} - \text{atraso do filtro} - 1$$

Esse ajuste está implementado em `PluckedString::new`, e o projeto tem um teste automatizado (`loop_delay_is_close_to_the_period`) que verifica, toda vez que o código muda, que a nota A4 realmente sai a menos de **1,5 cents** de 440 Hz.

CENTS. A unidade que músicos usam para medir desafinação bem fina. Um semitom (o menor “degrau” do piano, capítulo 3) vale 100 cents. 1,5 cents é uma fração tão pequena de um semitom (1,5%) que praticamente nenhum ouvido humano — nem afinadores profissionais, na maioria dos casos — consegue perceber a diferença. É esse o padrão de precisão que o projeto garante, e verifica automaticamente, para cada nota.

Cordas de verdade são rígidas: a inarmonicidade

No capítulo 5 dissemos que os harmônicos de uma corda ficam em múltiplos exatos da fundamental: $2\times$, $3\times$, $4\times$ etc. Isso é verdade para uma corda **ideal** — perfeitamente flexível, sem nenhuma resistência a dobrar. Cordas de piano de verdade, porém, são feitas de **aço grosso e razoavelmente rígido** (principalmente nas notas graves), e essa rigidez faz os harmônicos ficarem **ligeiramente mais agudos** do que os múltiplos exatos previstos — um efeito chamado **inarmonicidade**, descrito matematicamente pela primeira vez pelo físico Harvey Fletcher em 1964.

INARMONICIDADE. O quanto os harmônicos de um instrumento real se desviam de serem múltiplos exatos da fundamental, por causa da rigidez física da corda. É pequena, mas real — e é uma das razões pelas quais um piano de verdade soa “vivo” e um bipe eletrônico perfeitamente harmônico soa “morto” ou “artificial”.

A fórmula de Fletcher para onde cada harmônico realmente cai é:

$$f_n \approx n f_1 \sqrt{1 + Bn^2}$$

Não é preciso decorar a fórmula — o que ela diz, em palavras, é: “o harmônico número n fica um pouquinho mais agudo do que n vezes a fundamental, e esse desvio cresce com o quadrado de n (então harmônicos mais altos desviam proporcionalmente mais), multiplicado por um número B que descreve o quão rígida é aquela corda específica”. Cordas grossas e curtas (as graves) têm um B maior; cordas finas e longas (as agudas) têm um B menor.

Este é, aliás, o motivo pelo qual afinadores de piano profissionais não afinam o instrumento em oitavas *matematicamente* exatas — eles “esticam” ligeiramente as oitavas graves e agudas para compensar a inarmonicidade e fazer o piano soar afinado *aos ouvidos*, não apenas no papel. Esse fenômeno, bem documentado na literatura de acústica musical, é a mesma razão física que este projeto reproduz.

Como isso vira código

Reproduzir esse efeito dentro da “fila de espera” do capítulo 7 usa uma peça chamada **cascata de filtros allpass** (`piano_core::dispersion::DispersionCascade`), publicada por David Jaffe e Julius O. Smith em 1983.

FILTRO ALLPASS. Ao contrário do filtro passa-baixa do capítulo 8 (que muda o *volume* de diferentes frequências), um filtro allpass deixa o volume de **todas** as frequências exatamente igual — mas atrasa cada frequência por um tempo ligeiramente diferente. Isso é exatamente o que “esticar” os harmônicos para fora da posição exata significa: cada harmônico chega um pouquinho “fora de sincronia” com onde estaria numa corda ideal, sem que nenhum fique mais alto ou baixo em volume.

Encadear vários desses filtros (“uma cascata”) dentro do laço da corda aproxima, com boa precisão, a curva de inarmonicidade real. Quantos filtros são necessários varia por registro — cordas graves precisam de mais seções (cerca de 8 para a nota mais grave do piano) e cordas agudas precisam de quase nenhuma (0 a 1 para as notas mais agudas), porque o efeito de inarmonicidade é fisicamente mais forte nas cordas grossas e curtas do que nas finas e longas. Usar sempre o número máximo de seções, mesmo onde o ouvido não notaria diferença, gastaria processador à toa — por isso o projeto escala esse número por registro (documentado como decisão de performance `PERF-005`).

O martelo de feltro: como uma martelada vira som

A té agora falamos do que acontece **depois** que a corda já está vibrando. Falta a pergunta mais básica: **como a vibração começa?**

A técnica clássica de Karplus e Strong (1983), da qual este projeto parte, usa uma solução simples: enche a fila de espera inteira com **ruído aleatório**.

Ruído. Um sinal que contém, em proporções parecidas, todas as frequências ao mesmo tempo — como a “estática” de uma TV fora de sintonia; matematicamente é uma forma prática de “acender” todos os harmônicos de uma corda de uma vez.

Fisicamente, isso corresponde a “empurrar a corda inteira para uma posição aleatória de uma vez” — o que é uma boa aproximação para um instrumento **beliscado**, como um violão. Mas um piano não é beliscado, é **golpeado por um martelo de feltro**, e é exatamente esse detalhe que faz o piano soar como piano.

Por que a força da martelada muda o timbre, não só o volume

Toque uma tecla de piano bem de leve, depois toque a mesma tecla com toda a força. Você não vai ouvir “o mesmo som, só que mais alto” — vai ouvir um som mais **brilhante**, com mais agudos, quando toca forte. Esse é um dos traços mais característicos de um piano de verdade, e a razão física é o comportamento do próprio feltro do martelo.

O feltro que cobre o martelo funciona como uma mola, mas uma **mola não-linear**: quanto mais você a comprime, mais dura ela fica (diferente de uma mola comum, cuja dureza não muda). Esse comportamento foi medido e descrito por Antoine Chaigne e Anders Askenfelt em 1994, com a fórmula:

$$F = K x^p \quad (\text{com } p \text{ entre 2 e 3, aproximadamente})$$

Em palavras: a força F que o martelo exerce sobre a corda cresce **mais rápido que proporcionalmente** conforme a compressão x do feltro aumenta (por isso o expoente p é maior que 1 — se fosse $p = 1$, dobrar a compressão dobraria a força; com p entre 2 e 3, dobrar a compressão multiplica a força por 4 a 8 vezes). Uma martelada forte comprime mais o feltro, o que o deixa efetivamente mais “duro” durante aquele contato específico — um feltro mais duro transmite um golpe **mais curto e mais brusco**, e um golpe mais brusco excita harmônicos mais agudos com mais força. Uma martelada fraca comprime pouco, o feltro fica “macio”, o golpe é mais longo e suave, e o som sai mais quente, com menos agudos.

Este projeto implementa esse comportamento em `piano_core::hammer::simulate_contact`: dada a velocidade com que a tecla foi pressionada, o código simula o contato do martelo com a mola não-linear de feltro e produz um **envelope de força** (a forma da martelada ao longo do tempo, que dura menos de 5 milésimos de segundo). Esse envelope é usado para *moldar* o ruído inicial que preenche a fila de espera — em vez de simplesmente “mais forte = mais alto”, a forma do ataque muda com a velocidade.

HONESTIDADE SOBRE O QUE AINDA FALTA AQUI. O modelo atual simula o lado do *martelo* dessa interação — como o feltro se comprime e empurra — mas, durante os poucos milissegundos de contato, ainda não simula a corda “empurrando de volta” o martelo em tempo real (o problema físico completo, chamado de contato acoplado). Isso é uma simplificação documentada de propósito (rastreada como `PERF-007` na documentação técnica), e é exatamente o item no topo da lista de melhorias futuras do capítulo 17.

Mais de uma corda por tecla: uníssono e batimento

AQUI vai um fato que surpreende muita gente: **a maioria das teclas de um piano não tem apenas uma corda** — tem duas ou três, todas afinadas quase (mas não exatamente) na mesma frequência, e um único martelo bate nas cordas de uma tecla ao mesmo tempo. Um piano moderno típico tem:

- **1 corda** por tecla nas notas mais graves (mais grossas, mais caras, mais espaço ocupado — não cabem duas);
- **2 cordas** por tecla na região média-grave;
- **3 cordas** por tecla em toda a região média-aguda e aguda.

Este projeto usa essa mesma convenção — 12 teclas com 1 corda, 18 com 2 e 58 com 3 — o que soma **222 cordas efetivas** sendo processadas ao vivo, mesmo com só 88 teclas.

UNÍSSONO. (*unison*) O grupo de 1 a 3 cordas que uma única tecla controla. As cordas de um mesmo uníssono são afinadas *quase* iguais, mas não perfeitamente — normalmente há uma diferença de poucos “cents” (capítulo 9) entre elas, de propósito.

Por que desafinar de propósito?

Se as duas ou três cordas de uma tecla fossem afinadas **exatamente** iguais, elas vibrariam perfeitamente em sincronia, e o resultado soaria idêntico a uma única corda — apenas mais alto. Mas quando estão afinadas com uma diferença bem pequena, acontece um fenômeno físico chamado **batimento**:

BATIMENTO. (*beating*) Quando duas ondas sonoras de frequências muito próximas (mas não idênticas) tocam juntas, elas alternadamente se reforçam e se cancelam, produzindo um efeito de “pulsar” no volume — um “uauauauau” lento, cuja velocidade é exatamente igual à diferença entre as duas frequências. Você pode ouvir isso muito claramente quando duas cordas de violão quase afinadas tocam juntas: em vez de um som estável, ouve-se um “bater” rítmico que vai sumindo conforme se ajusta a afinação.

Esse “bater” é parte do que dá a um piano de verdade seu caráter rico e “vivo”, em vez de um som mais “seco” e sintético.

O envelope de dois estágios

Tem mais uma consequência física interessante, medida e descrita formalmente pelo físico Gabriel Weinreich em 1977: as cordas de um mesmo uníssono não vibram de forma independente — elas estão mecanicamente conectadas através da ponte (capítulo 4), que embora seja rígida, cede um pouquinho. Essa conexão parcial faz o som de uma nota de piano decair em **dois estágios**, não um só:

1. **Pré-decaimento** (rápido): logo após a martelada, as cordas ligeiramente desafinadas batem entre si e perdem energia relativa umas às outras rapidamente — essa fase soa mais “cheia” e um pouco instável.
2. **Cauda** (mais lenta): depois que essa energia diferencial se dissipa, as cordas se estabilizam num modo compartilhado que decai numa taxa parecida com a de uma única corda “natural” — essa é a fase mais longa e estável, o “sustain” que você ouve segurando a tecla.

Este projeto implementa exatamente esse mecanismo (não um envelope artificial “desenhado à mão” para parecer parecido) em duas camadas:

- **Acoplamento local** (`piano_core::unison`): as 1 a 3 cordas da mesma tecla se misturam entre si a cada amostra individual — é barato de calcular porque essas cordas já são processadas juntas de qualquer forma.
- **Acoplamento global** (`piano_core::bridge::BridgeBus`, capítulo 13): a interação entre teclas *diferentes*, através da ponte compartilhada de todo o piano.

O projeto tem inclusive um teste automatizado que **mede** esse efeito (não apenas confia que o código “deveria” produzir isso): ele renderiza uma nota de três cordas, mede a velocidade do decaimento logo após o ataque comparada com a velocidade depois que o batimento já se estabilizou, e confirma que a primeira é visivelmente mais rápida que a segunda — a assinatura exata que o modelo de Weinreich prevê. Esse teste, durante o desenvolvimento, pegou dois bugs reais de implementação antes que chegassem ao código final — um exemplo de como medir o comportamento físico é mais confiável do que apenas ler o código e “achar” que está certo.

O pedal mágico: ressonância simpática

SENTE-SE ao piano, pise no pedal direito (sustentação) sem tocar nenhuma tecla, e cante ou grite uma nota perto das cordas. Você vai ouvir o piano “responder” — alguma corda dentro dele começa a vibrar sozinha, mesmo sem ninguém tê-la tocado. Isso é **ressonância simpática**, e é um dos efeitos mais mágicos (e mais difíceis de simular bem) de um piano de verdade.

RESSONÂNCIA SIMPÁTICA. Quando um objeto vibrante (uma corda, uma voz) faz *outro* objeto próximo, capaz de vibrar na mesma frequência ou numa relacionada, começar a vibrar também — sem nenhum contato direto, só através do ar ou de uma estrutura compartilhada. É o mesmo princípio por trás de empurrar um balanço no ritmo certo para fazê-lo ir cada vez mais alto: pequenos empurrões, no momento certo, se acumulam.

Num piano de verdade, isso acontece porque **todas as cordas do instrumento** estão fisicamente conectadas através da mesma ponte e da mesma caixa de ressonância. Normalmente, os abafadores de feltro mantêm as cordas que você não está tocando “mudas” — mas quando você pisa no pedal direito, **todos os abafadores levantam ao mesmo tempo**, e qualquer corda cuja frequência (ou harmônico) combine com o que está soando pode “pegar carona” na vibração e começar a soar também, mesmo sem ter sido golpeada.

Como reproduzir isso sem explodir o custo computacional

A forma fisicamente mais completa de simular isso seria calcular como **cada uma das 222 cordas efetivas afeta todas as outras 221**, individualmente — o que dá mais de 49 mil pares de interação, recalculados a cada amostra, 48 mil vezes por segundo. Isso é computacionalmente inviável em tempo real, mesmo em computadores potentes.

A solução do projeto (`piano_core::bridge::BridgeBus`) é mais elegante: todas as cordas escrevem sua vibração num único **barramento compartilhado** — pense nele como um “balde” onde cada corda despeja um pouco do seu som, e de onde cada corda também lê uma média de tudo que está no balde. Isso captura o essencial do efeito real (cordas “sentem” o que as outras estão fazendo) com um custo que cresce de forma administrável — em vez de crescer explosivamente com o número de cordas.

SIMPLIFICAÇÃO, DITA COM HONESTIDADE. O barramento único usa o **mesmo** ganho de acoplamento para todas as cordas, e não modela como a ponte de um piano real transmite som de forma diferente dependendo de onde, na madeira, cada corda está apoiada — isso exigiria medições de um instrumento físico específico, algo que o projeto delibera não fazer sem antes decidir explicitamente que fontes de dado são aceitáveis (capítulo 17). É uma aproximação de engenharia, declarada abertamente na documentação técnica, não apresentada como mais fiel do que realmente é.

A caixa de ressonância: de onde vem o volume

Como mencionado no capítulo 4, uma corda de piano sozinha é fininha demais para empurrar ar suficiente e ser ouvida com volume real — quase todo o som que você de fato escuta vem da **caixa de ressonância**, o grande painel de madeira que a ponte transmite a vibração para.

A forma mais “fiel à física” de simular uma caixa de ressonância seria gravar, de um piano real, como ele responde a um impulso curtíssimo (um “clique”), e depois combinar matematicamente (“convoluir”) essa gravação com o som de cada corda. **Este projeto não pode fazer isso**, e não por falta de capacidade técnica: a regra fundamental do projeto proíbe qualquer gravação de áudio real, para sempre, em qualquer circunstância — todo o som precisa nascer de cálculo, nunca de amostras gravadas.

A alternativa escolhida é a **síntese modal**: representar a caixa de ressonância como um conjunto de “modos” — frequências específicas nas quais a madeira naturalmente prefere vibrar, cada uma com seu próprio tempo de decaimento e volume relativo, baseados em valores típicos descritos na literatura acadêmica de acústica (não medidos de nenhum piano específico).

MODO DE RESSONÂNCIA. Assim como uma corda tem harmônicos (capítulo 5), uma placa de madeira grande também tem frequências preferenciais nas quais ela “gosta” de vibrar quando excitada — mas, ao contrário de uma corda (onde os harmônicos formam uma série organizada de múltiplos), os modos de uma placa bidimensional são mais irregulares e dependem do formato e material específicos da placa.

O projeto usa 8 desses modos (`piano_core::soundboard::Soundboard`), somados ao sinal direto de cada corda (não substituindo-o) antes de virar som final — porque substituir completamente mudaria a afinação e o brilho medidos e testados em capítulos anteriores, uma troca (*trade-off*) que o projeto documenta abertamente em vez de mudar silenciosamente.

Por que o programa nunca pode travar

LEMBRA do capítulo 6? A cada $\frac{1}{48000}$ de segundo (cerca de 20,8 microssegundos), o programa precisa ter calculado o próximo número de áudio, para **todas** as vozes tocando simultaneamente, e entregá-lo à placa de som. Se ele demorar mais que isso mesmo uma única vez, o resultado não é “um pouquinho de atraso” — é um **estalo, clique ou silêncio audível** na música, porque a placa de som não tem o que tocar naquele instante exato.

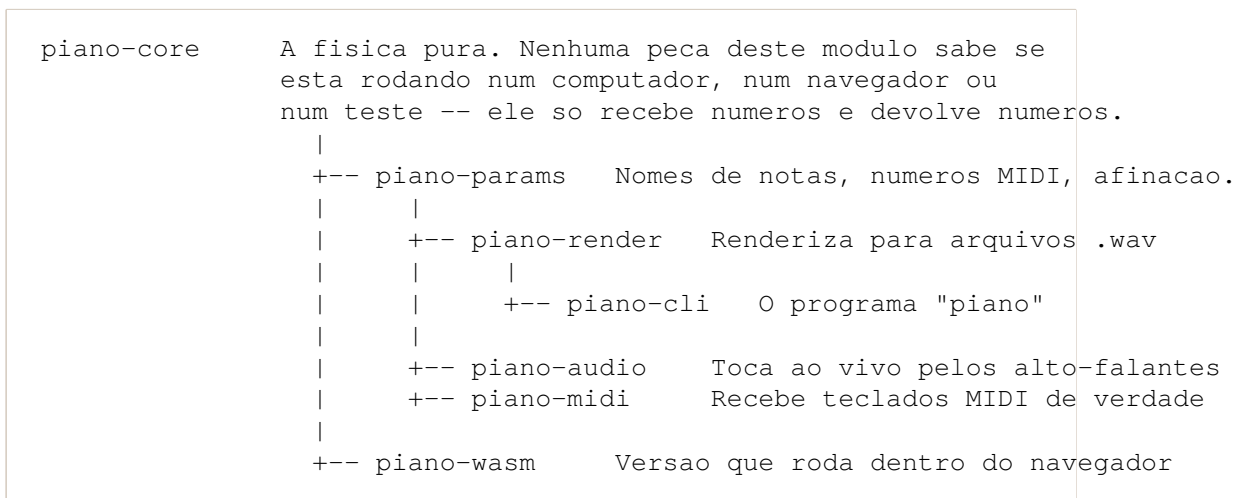
Isso impõe uma regra muito mais rígida do que a maioria dos programas de computador precisa seguir. A parte do código que calcula o áudio (chamada de **thread de tempo real**, porque roda numa linha de execução separada e dedicada exclusivamente a isso) segue quatro regras absolutas, verificadas automaticamente a cada mudança no código:

1. **Nunca aloca memória nova.** Pedir memória nova ao sistema operacional pode, ocasionalmente, demorar um tempo imprevisível — inaceitável dentro da janela de 20,8 microssegundos. Toda a memória que uma nota vai precisar é reservada com antecedência, na hora em que o programa começa a rodar, não quando você aperta a tecla.
2. **Nunca espera por outra parte do programa** (nenhum “cadeado” ou trava de sincronização). Se essa parte do código tivesse que esperar outra parte terminar algo, e essa outra parte estivesse, por qualquer motivo, atrasada, o áudio inteiro travaria com ela.
3. **Nunca pode gerar um erro fatal.** Um programa comum, ao encontrar uma situação inesperada, pode simplesmente parar com uma mensagem de erro. Isso é inaceitável no meio de uma peça musical — então todo pedaço de código nessa parte do projeto é escrito de um jeito que **sempre** devolve algum resultado válido, mesmo em casos extremos (um número “infinito”, um número inválido, zero etc.).
4. **Nunca entra num laço sem fim garantido.** Todo cálculo tem um número máximo de passos conhecido de antemão — nada de “repita até algo acontecer” sem um limite garantido.

O projeto até usa uma ferramenta do próprio compilador da linguagem Rust (chamada `no_std`, e uma configuração que **proíbe** o uso das funções `unwrap`, `expect` e `panic!` fora de testes) para tornar **impossível**, não apenas malvisto, escrever código que quebre essas regras sem querer — o próprio compilador recusa compilar o programa se alguém tentar. Isso é o que separa “um programa que geralmente não trava” de “um programa que matematicamente não pode travar por esses motivos”.

Como as peças do projeto se encaixam

Tudo o código está organizado em módulos independentes (chamados, na linguagem Rust, de **crates**), cada um com uma única responsabilidade clara, para que uma mudança numa parte não possa quebrar outra por acidente:



A ideia central — repetida por todo o projeto — é: **o motor de física (piano-core) não sabe nada sobre o mundo exterior**. Ele não sabe se está sendo tocado por um teclado de computador, por um piano MIDI de verdade conectado por USB, ou por uma página de navegador. Só as camadas *fora dele* (piano-audio, piano-midi, piano-wasm, piano-cli) sabem lidar com o “mundo real” — placas de som, cabos USB, páginas web. Isso é o que permite o **mesmo** código de física rodar de forma idêntica em três lugares completamente diferentes, sem nenhuma parte especial para cada um.

O que ainda falta para chegar perto de um piano de verdade

ESTE projeto não é um protótipo incompleto — cada mecanismo descrito nos capítulos 7 a 14 já está implementado, testado e verificado (não apenas planejado). Mas um piano de verdade tem ainda mais detalhes do que os já cobertos, e este capítulo lista os que faltam, honestamente, do maior impacto perceptível para o menor:

O que falta	Por que importa	Marco
Contato martelo-corda totalmente acoplado	Hoje o martelo (capítulo 11) empurra a corda, mas a corda ainda não “empurra de volta” o martelo durante a martelada — a versão completa muda sutilmente o ataque de cada nota	M9
Posição da martelada e curvas por tecla	Onde exatamente o martelo bate ao longo da corda muda o timbre; hoje isso ainda não varia por nota	M9
Pedal una corda (esquerdo) e sostenuto (do meio)	Hoje só existe o pedal de sustentação (direito); um piano de verdade tem três pedais, cada um com um efeito diferente	M10
Ressonância simpática mais rica	O barramento único (capítulo 13) já funciona, mas uma versão mais fiel diferenciaria o quanto cada corda “sente” as outras	M11
Caixa de ressonância mais detalhada	8 modos (capítulo 14) já dão corpo ao som; um banco maior aproximaria ainda mais a complexidade real da madeira	M11
Ruídos mecânicos	Um piano de verdade não é só corda — o mecanismo em si faz ruído, sempre sintetizado, nunca gravado	M12
Modelar um piano específico de verdade	Hoje o projeto usa números “típicos” da literatura, não medidos de um instrumento físico específico; exigiria decidir, antes de qualquer código, que dado medido é aceitável	M13
Um plugin de estúdio (CLAP)	Permitiria usar o piano dentro de programas como Ableton, Logic ou Reaper	M14

Esses marcos (M9 a M14) já existem como **issues públicas no GitHub** do projeto, para quem quiser acompanhar ou contribuir — não são apenas ideias soltas, são trabalho planejado e rastreável.

Vale reforçar: mesmo com toda essa lista de “o que falta”, o piano **já funciona de verdade hoje** — os capítulos 7 a 14 não são teoria, são o comportamento real do programa que você pode compilar e tocar agora mesmo (veja `docs/pt-BR/LEIA-ME.md` para o passo a passo de instalação e uso).

Tocando de verdade: como o MIDI leva sua martelada até o código

TUDO que os capítulos anteriores descreveram — a fila de espera, o filtro, a martelada de feltro — precisa de uma entrada: alguém precisa dizer ao programa *qual* tecla foi tocada, *com que força*, e *quando*. Este capítulo explica como essa informação chega até o código, usando exatamente o protocolo que qualquer piano digital ou teclado controlador de verdade fala: o **MIDI**.

MIDI não é som — é uma partitura tocada em tempo real

MIDI. (*Musical Instrument Digital Interface*) Um protocolo de 1983 para instrumentos musicais eletrônicos conversarem entre si, transportando *eventos de performance* — “esta tecla foi pressionada”, “esta tecla foi solta”, “este pedal desceu” — e nunca áudio propriamente dito. É a diferença entre uma partitura (o que tocar, quando, com que intensidade) e uma gravação (o som já pronto): MIDI é sempre a partitura, nunca a gravação.

Isso importa duas vezes para este projeto: primeiro, é assim que uma martelada real, tocada num teclado de verdade, chega até `piano_core::hammer::simulate_contact` (capítulo 11); segundo, explica por que o teclado físico plugado por USB pode ser de qualquer marca — ele só precisa falar essa mesma língua comum, definida em 1983 e praticamente inalterada desde então em todo teclado musical fabricado hoje.

As três mensagens que este projeto entende

O código que decodifica MIDI neste projeto (`piano_midi::event::parse`) reconhece só três tipos de mensagem — deliberadamente, porque só três são necessários para o que o instrumento faz hoje:

Mensagem MIDI	O que dispara no código	Onde
Note On	<code>AudioSession::note_on(nota, velocidade)</code>	tecla pressionada
Note Off	<code>AudioSession::note_off(nota)</code>	tecla solta
Control Change (CC)	muda um parâmetro ao vivo	knob, pedal, roda

Dentro de “Control Change”, três números de controlador específicos já têm um significado neste projeto (`piano-cli/src/midi.rs`):

CC	Nome padrão MIDI	O que faz aqui
74	Brightness	damping, invertido: girar para cima escurece
1	Modulation wheel	sustain (capítulo 8)
64	Sustain (hold) pedal	segura toda nota solta até o pedal subir

POR QUE CC74 E CC1, E NÃO ALGO DEDICADO. O padrão MIDI define um número fixo de controladores “com nome” (*brightness, modulation wheel...*), mas nenhum deles se chama “quanto uma corda física decai”. Este projeto reaproveita o controlador padrão mais parecido em intenção — CC74 já é, por convenção, o “deixa mais brilhante ou mais escuro” de quase todo sintetizador — em vez de inventar um número de controlador só seu, que nenhum teclado físico teria um knob já mapeado para controlar.

Por que a velocidade é sempre um número entre 0 e 127

Toda vez que este projeto normaliza uma velocidade ou um valor de CC, ele divide por 127 (`event::normalize`, capítulo anterior a este). Esse 127 não é um limite deste projeto — é um limite do **protocolo MIDI em si**: cada mensagem carrega o dado de velocidade num único byte de 7 bits, e 7 bits contam de 0 a $2^7 - 1 = 127$. Não existe, no MIDI 1.0 (o que `piano-midi` fala, via a biblioteca `midir`), um jeito de mandar “velocidade 200” ou “velocidade 128,5” — o teto é sempre exatamente 128 valores possíveis, não importa quão caro ou sofisticado seja o teclado do outro lado do cabo.

HONESTIDADE SOBRE O MIDI 2.0. Existe uma versão mais nova do protocolo, MIDI 2.0 (2020), que permite velocidade de até 32 bits por nota — bilhões de valores, não 128. Praticamente nenhum piano digital no mercado (incluindo o deste capítulo) fala MIDI 2.0 hoje, e `piano-midi` também não — então, na prática, o teto de 128 continua sendo o que qualquer combinação de hardware e software deste projeto realmente usa.

Exemplo prático: o alcance de um piano digital convencional

Para tornar isso concreto em vez de abstrato, vale examinar um piano digital real, de entrada de linha, e ver exatamente onde ele para: o **Yamaha P-125**, um dos pianos digitais de 88 teclas mais vendidos do mundo nessa faixa de preço.

O P-125 usa a ação **GHS** (*Graded Hammer Standard*) da Yamaha: 88 teclas com peso graduado — mais pesadas nas graves, mais leves nas agudas, imitando a distribuição de peso dos martelos reais de um piano acústico — construída com **dois sensores por tecla** (a diferença de tempo entre os dois sensores é o que o firmware do teclado usa para calcular a velocidade que sai como MIDI), sem mecanismo de escapamento e sem teclas de madeira. O teclado ainda oferece quatro curvas de sensibilidade ao toque (desligada, suave, média, forte) — que *remapeiam* os mesmos 128 valores possíveis em curvas diferentes, não aumentam quantos valores existem.

“Pianos mais caros têm mecanismos mais precisos para enviar MIDI? Isso é verdade?”

Sim — mas não do jeito que a pergunta sugere à primeira vista. O número que sai pelo cabo **nunca** passa de 128 valores, custe o piano \$500 ou \$50.000: isso é um teto do protocolo (seção anterior), não do hardware. O que muda com o preço é *outra coisa*: quão bem o mecanismo consegue capturar o gesto real do pianista — de forma consistente, tecla a tecla, em toda a extensão do teclado — e converter isso nesse mesmo intervalo fixo de 128 valores, além de que *tipos* de mensagem o teclado consegue produzir.

Ação	Sensores/tecla	Escapamento	Teclas	<i>Aftertouch</i>
GHS (P-125)	2	não	plástico	não — falta o sensor
GH3	3	não	plástico	sim
GH3X	3	sim	plástico	sim
NWX	3	sim	madeira (brancas)	sim

POR QUE O TERCEIRO SENSOR IMPORTA. *Aftertouch* (pressão contínua depois que a tecla já chegou ao fundo) exige um terceiro ponto de contato perto do fim do curso da tecla. Numa ação de dois sensores como a GHS, esse terceiro ponto **não existe fisicamente** — não é uma limitação de firmware que uma atualização resolveria, é a mecânica em si que não tem como detectar isso. O escapamento (a “marchinha” que você sente na metade do curso de um piano acústico de verdade) e as teclas de madeira, por sua vez, não mudam nenhum número que sai pelo MIDI — elas mudam o quão bem o *pianista* consegue controlar a força com que aperta a tecla, o que indiretamente produz uma velocidade mais consistente e mais expressiva dentro do mesmo teto de 128.

A capacidade do seu Yamaha P-125, especificamente

Característica	Valor
Teclas	88, com ação GHS (martelo graduado, 2 sensores)
Curvas de toque	4 predefinidas: desligada, suave, média, forte
<i>Aftertouch</i>	Não suportado (ação sem terceiro sensor)
Saída MIDI	USB-to-Host (mesmo cabo de energia de dados)
Pedal incluso	Liga/desliga simples (sustain, sem meio-pedal)
Meio-pedal	Só com o pedal opcional Yamaha FC3A (contínuo)
Motor de som interno	Pure CF (amostras do CFIIS), 192 vozes de polifonia

Como o P-125 faz o próprio som dele — e por que isso não importa aqui

Vale fechar o círculo numa pergunta natural: se o P-125 já soa como um piano, como ele faz isso? A resposta é **o oposto exato** da técnica deste projeto. O motor de som do P-125, chamado pela Yamaha de “Pure CF Sound Engine”, é baseado em **amostras**: gravações reais do piano de cauda de concerto Yamaha CFIIS, tocadas de volta quando você aperta uma tecla, com camadas diferentes de gravação conforme a velocidade da martelada e até 192 notas soando ao mesmo tempo (a polifonia). É literalmente a técnica que o capítulo 1 e a raiz deste repositório (CLAUDE.md) descartam de propósito para `piano_core`: áudio gravado, reproduzido de volta.

POR QUE ISSO NÃO É UMA CONTRADIÇÃO. Usar o P-125 como controlador para este projeto não usa nenhuma amostra do P-125 — só o teclado (a mecânica GHS) é aproveitado, para gerar mensagens MIDI. O alto-falante e o motor de som Pure CF do P-125 ficam completamente fora do circuito; o som que sai é 100% calculado por `piano_core`, a partir da física do capítulo 7 em diante. O P-125, nesse uso, é só um sensor de toque sofisticado — a mesma função que um teclado de computador (capítulo 5 de docs/pt-BR/LEIA-ME.md) cumpre de um jeito bem mais grosseiro, sem velocidade nenhuma.

Honestidade: o que este projeto ainda não faz com o MIDI que recebe

Para não prometer mais do que o código entrega hoje:

- ***Aftertouch, pitch bend e mudança de programa são descartados silenciosamente*** — `piano_midi::event::parse` devolve `None` para eles, por design (são mensagens que este instrumento simplesmente não usa ainda, não um bug). Mesmo com um teclado que os produzisse (o P-125 não produz *aftertouch*, tabela acima), este projeto os ignoraria hoje.
- **O pedal de sustentação é liga/desliga, não contínuo** — o código compara o valor do CC64 com um limiar fixo (`value >= 64.0/127.0`) em vez de usar o valor contínuo que o MIDI já carrega. Mesmo comprando o pedal opcional FC3A do P-125 (que envia um CC64 contínuo, não só 0 ou 127), este projeto ainda não usaria a nuance do meio-pedal.
- **inharmonicidade e a configuração do martelo (`HammerConfig`) não têm nenhum CC mapeado** — ao contrário de `damping` e `sustain`, não há hoje um jeito de ajustá-los ao vivo por um controlador MIDI (só escrevendo Rust diretamente). É o mesmo ponto que o capítulo anterior sobre timbre já registrou.

Glossário completo

Todas as palavras novas deste documento, num só lugar, para consulta rápida.

Termo	Significado
Onda	Uma perturbação que se propaga por um meio sem que o meio em si viaje junto.
Frequência Hertz (Hz)	Quantas vezes por segundo algo se repete, medida em hertz (Hz). Unidade de frequência: “vezes por segundo”.
Nota musical	Um som cuja frequência é constante e bem definida durante o tempo em que soa.
Oitava	O intervalo entre uma frequência e o dobro dela.
Semitom	O menor degrau entre notas num piano; 12 semitons formam uma oitava.
A4 / Lá4	Nota de referência do piano, fixada em 440 Hz.
Harmônico / parcial / overtone	Cada uma das vibrações simultâneas de uma corda, em múltiplos da fundamental.
Fundamental	O primeiro harmônico — a corda vibrando inteira, de ponta a ponta.
Timbre	A “cor” característica de um instrumento, determinada pela mistura de harmônicos.
Amostra (<i>sample</i>)	Um único número medindo a pressão do ar (ou posição do alto-falante) num instante.
Taxa de amostragem	Quantas amostras são geradas por segundo (48.000 neste projeto).
Linha de atraso (<i>delay line</i>)	Uma fila de números de tamanho fixo, onde um número novo entra e o mais antigo sai a cada passo.
Guia de onda digital	A técnica de representar uma corda vibrando como uma linha de atraso circulando.
Filtro passa-baixa	Um mecanismo que reduz frequências altas e deixa as baixas passarem quase sem mudança.
Amortecimento (<i>damping</i>)	O quanto de energia uma corda perde por unidade de tempo.
Cents	Unidade fina de afinação; 100 cents formam um semitom.
Inarmonicidade	O desvio dos harmônicos de uma corda real em relação a múltiplos exatos da fundamental, causado pela rigidez física da corda.
Filtro allpass	Um filtro que atrasa frequências diferentes por tempos diferentes, sem mudar o volume de nenhuma.
Ruído	Um sinal que contém várias frequências ao mesmo tempo, em proporções parecidas.
Uníssonos (<i>unison</i>)	O grupo de 1 a 3 cordas que uma mesma tecla de piano controla.
Batimento (<i>beating</i>)	O “pulsar” de volume que surge quando dois sons de frequências muito próximas tocam juntos.
Ressonância simpática	Quando um objeto vibrante faz outro, próximo, começar a vibrar também, sem contato direto.
Modo de ressonância	Uma frequência preferencial na qual uma estrutura naturalmente vibra.
Ponte (<i>bridge</i>)	A peça que transmite a vibração de uma corda para a caixa de ressonância.
Modelagem física	Descrever um objeto real como equações e resolvê-las em tempo real para produzir seu comportamento.
Sampler	Um instrumento que reproduz gravações prontas em vez de calcular o som.

Para saber mais

ESTE documento é a versão didática e sem pré-requisitos de material técnico mais denso que já existe no projeto, em inglês, para quem quiser ir mais fundo:

- **docs/PHYSICS.md** — o mapeamento completo, com mais detalhes técnicos e as referências acadêmicas exatas por trás de cada mecanismo descrito neste documento.
- **docs/ARCHITECTURE.md** — como o código está organizado por dentro.
- **docs/PERFORMANCE.md** — as medições reais de desempenho por trás de cada decisão de engenharia mencionada aqui.
- **docs/ROADMAP.md** — todos os marcos do projeto, passados e futuros.
- **docs/pt-BR/LEIA-ME.md** — o guia de instalação e uso em português, passo a passo, para quem quer simplesmente tocar o piano agora.
- Os artigos científicos originais citados ao longo deste documento (Karplus e Strong 1983; Jaffe e Smith 1983; Fletcher 1964; Chaigne e Askenfelt 1994; Weinreich 1977) estão listados, com título completo e onde encontrá-los, em **docs/PRIOR-ART.md**.

Se depois de ler tudo isso uma pergunta ainda ficou sem resposta, ela provavelmente merece virar uma *issue* no GitHub do projeto — é assim que esta documentação, e o próprio piano, continuam melhorando.